

Sprints (Updated with Enhanced Knowledge Base)

The enhanced **Knowledge Base Marketplace** tasks (uploader, packaging/download) are primarily added to **Sprint 3**, with integration tasks spread across Sprints 4-6 to ensure cohesiveness.

Sprint 1: Foundation and Core UI (2 Weeks)

Tasks:

- **1.1: Setup Replit Monorepo** (No changes; see previous artifact).
- **1.2: Build Express APIs for User Authentication** (No changes).
- **1.3: Create Component Library and UI** (No changes).
- **1.4: Configure Email for Welcome/Verification** (No changes). **Prompts:**
 - Same as previous artifact, ensuring monorepo, auth, UI, and email foundation.
- **Cohesiveness Notes:** Component library includes Uploader component for Sprint 3; auth supports marketplace sellers.

Sprint 2: Curriculum, Lessons, and Virtual Tutor (3 Weeks)

Tasks:

- **2.1: APIs for Curriculum/Lesson Generation** (Updated to reference uploaded materials).
- **2.2: Curriculum Engine and Tutor** (Updated to use uploaded materials).
- **2.3: Curriculum Dashboard, Lesson Viewer, and Tutor UI** (No changes).
- **2.4: Schedule and Progress Update Emails** (No changes). **Prompts:**
- **Task 2.1: APIs for Curriculum/Lesson Generation:**
 - **Description:** Create APIs to generate curricula/lessons, referencing uploaded materials.
 - **Files:**
 1. /ai/src/curriculum.js
 2. /server/src/routes/curriculum.js
 3. /server/src/models/knowledgeBase.js (new)
 - **Dependencies:**
 1. npm: axios
 2. Pip: transformers torch
 - **Steps:**

In /server, create /server/src/models/knowledgeBase.js:

javascript

Copy

```
const mongoose = require('mongoose');
const schema = new mongoose.Schema({
  title: String,
  creator: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  subject: String,
  difficulty: String,
```

```

    objectives: [String],
    files: [{ url: String, type: String, name: String }],
    metadata: { tags: [String] },
    price: Number,
    purchasedBy: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  });

```

```

1. module.exports = mongoose.model('KnowledgeBase', schema);

```

Update /ai/src/curriculum.js:

javascript

Copy

```

const { spawn } = require('child_process');
const path = require('path');
const KnowledgeBase = require('../../../../server/src/models/knowledgeBase');

const generateCurriculum = async (profile) => {
  const bases = await KnowledgeBase.find({ subject: { $in: profile.subjects } });
  const python = spawn('python', [path.join(__dirname, 'curriculum.py'),
JSON.stringify({ ...profile, bases })]);
  let output = '';
  python.stdout.on('data', (data) => output += data);
  python.stderr.on('data', (data) => reject(data.toString()));
  python.on('close', () => resolve(JSON.parse(output)));
};

```

```

2. module.exports = { generateCurriculum };

```

Update /ai/src/curriculum.py:

python

Copy

```

import sys
import json
from transformers import pipeline

def generate_curriculum(profile):
    nlp = pipeline('text-generation', model='distilgpt2')
    files = [f for base in profile['bases'] for f in base['files']]
    prompt = f"Generate a curriculum for a {profile['age']} year old using
files: {files}"
    curriculum = nlp(prompt, max_length=100)[0]['generated_text']
    return {'curriculum': curriculum, 'lessons': [{ 'id': 1, 'title': 'Intro',
'content': files[0]['url'] if files else 'Sample' }]}

```

```
if __name__ == '__main__':
    profile = json.loads(sys.argv[1])
    result = generate_curriculum(profile)
```

```
3.      print(json.dumps(result))
```

4. Update /server/src/routes/curriculum.js (as in previous artifact).

5. Test API in Replit (POST /api/curriculum/generate, body: { "age": 12, "subjects": ["math"], "email": "test@example.com" }).

- **Expected Outcome:** Curriculum API generates lessons referencing uploaded materials.
- **Cohesiveness Notes:** Knowledge base model supports uploader; integrates with tutor.

- **Task 2.2: Curriculum Engine and Tutor:**

- **Description:** Enhance curriculum engine and tutor to use uploaded materials.

- **Files:**

1. /ai/src/engine.py
2. /ai/src/tutor.js

- **Dependencies:**

1. Pip: tensorflow
2. npm: axios

- **Steps:**

Update /ai/src/engine.py:

python

Copy

```
import tensorflow as tf
import json
import sys
```

```
def curriculum_engine(profile):
    bases = profile.get('bases', [])
    lessons = [{ 'id': 1, 'title': f"{profile['subjects'][0]} Intro",
'content': bases[0]['files'][0]['url'] if bases else 'Sample' }]
    return lessons
```

```
if __name__ == '__main__':
    profile = json.loads(sys.argv[1])
    result = curriculum_engine(profile)

1.      print(json.dumps(result))
```

Update /ai/src/tutor.js:

javascript

Copy

```
const generateTutorResponse = async (question, context, baseFiles) => {
```

```

    const prompt = `Context: ${context}. Files: ${baseFiles.join(', ')}.\nQuestion: ${question}.`;
    return await callClaudeAPI(prompt);
  };

```

```

2. module.exports = { generateTutorResponse };

```

Update /server/src/routes/curriculum.js:

javascript

Copy

```

router.post('/tutor', async (req, res) => {
  const { question, context, baseId } = req.body;
  const base = await KnowledgeBase.findById(baseId);
  const response = await generateTutorResponse(question, context, base ?
base.files.map(f => f.url) : []);
  res.json({ response });

```

```

3. });

```

```

4. Test tutor API in Replit (POST /api/curriculum/tutor, body: { "question":
"Explain algebra", "context": "Math lesson", "baseId": "base_id" }).

```

- **Expected Outcome:** Tutor uses uploaded materials for responses.
- **Cohesiveness Notes:** Extends curriculum; reusable for camps.
- **Tasks 2.3-2.4:** No changes (as in previous artifact).

Sprint 3: Knowledge Base Marketplace and Social Community (3 Weeks)

Tasks:

- **3.1: GraphQL for Marketplace/Community Queries** (Updated to support packaged bases).
- **3.2: Marketplace and Community APIs** (Updated for uploader and packaging).
- **3.3: Blockchain for Badges** (Updated to include knowledge base licensing).
- **3.4: Marketplace and Community UI** (Updated for uploader and download).
- **3.5: Purchase and Community Emails** (Updated for download notifications). **Prompts:**
- **Task 3.1: GraphQL for Marketplace/Community Queries:**
 - **Description:** Enhance GraphQL schema for packaged knowledge bases.
 - **Files:**
 1. /server/src/schema.js
 2. /server/src/resolvers.js
 - **Dependencies:** None (uses existing @apollo/server).
 - **Steps:**

Update /server/src/schema.js:

javascript

Copy

```

const typeDefs = gql`
  type KnowledgeBase {

```

```

    id: ID!
    title: String!
    creator: String!
    subject: String!
    price: Float!
    files: [File]
    metadata: Metadata
  }
  type File { url: String, type: String, name: String }
  type Metadata { tags: [String] }
  type CommunityPost { id: ID!, content: String!, author: String! }
  type Query { knowledgeBases: [KnowledgeBase], communityPosts:
[CommunityPost] }
  type Mutation {
    createKnowledgeBase(title: String!, creator: String!, subject: String!,
price: Float!, files: [FileInput], metadata: MetadataInput): KnowledgeBase
    createCommunityPost(content: String!, author: String!): CommunityPost
    purchaseKnowledgeBase(id: ID!, userId: ID!): KnowledgeBase
  }
  input FileInput { url: String, type: String, name: String }
  input MetadataInput { tags: [String] }
`;

```

1. `module.exports = typeDefs;`

Update `/server/src/resolvers.js`:

javascript

Copy

```

const purchaseKnowledgeBase = async (_, { id, userId }) => {
  const base = await KnowledgeBase.findById(id);
  base.purchasedBy.push(userId);
  await base.save();
  // Trigger blockchain licensing (Task 3.3)
  return base;
};

const Mutation = {
  createKnowledgeBase: async (_, { title, creator, subject, price, files,
metadata }) => {
    const base = new KnowledgeBase({ title, creator, subject, price, files,
metadata });
    await base.save();
    return base;
  },
  createCommunityPost,
  purchaseKnowledgeBase,

```

2. };

3. Test GraphQL in Replit's preview (<http://localhost:4000>).

- **Expected Outcome:** GraphQL supports packaged bases with files/metadata.
- **Cohesiveness Notes:** Extends marketplace; integrates with uploader.
- **Task 3.2: Marketplace and Community APIs:**
 - **Description:** Add APIs for uploading materials, packaging bases, and downloading.
 - **Files:**
 1. /server/src/routes/upload.js
 2. /server/src/services/packager.js
 - **Dependencies:**
 1. npm: multer archiver aws-sdk
 - **Steps:**
 1. In /server, run npm install multer archiver aws-sdk.
 2. Update /server/.env:

```
plaintext
Copy
AWS_S3_BUCKET=your_s3_bucket
```

Create /server/src/routes/upload.js:

javascript

Copy

```
const express = require('express');
const multer = require('multer');
const AWS = require('aws-sdk');
const KnowledgeBase = require('../models/knowledgeBase');
const router = express.Router();

const s3 = new AWS.S3();
const upload = multer({ dest: 'uploads/' });

router.post('/upload', upload.array('files'), async (req, res) => {
  const files = req.files.map(file => ({
    url:
`https://${process.env.AWS_S3_BUCKET}.s3.amazonaws.com/${file.filename}`,
    type: file.mimetype,
    name: file.originalname,
  }));
  for (const file of req.files) {
    await s3.upload({
      Bucket: process.env.AWS_S3_BUCKET,
      Key: file.filename,
      Body: require('fs').createReadStream(file.path),
    }).promise();
  }
}
```

```

    const base = new KnowledgeBase({ ...req.body, files });
    await base.save();
    res.json(base);
  });

router.get('/download/:baseId', async (req, res) => {
  const base = await KnowledgeBase.findById(req.params.baseId);
  const zip = require('archiver')('zip');
  res.attachment(`${base.title}.zip`);
  zip.pipe(res);
  for (const file of base.files) {
    const { Body } = await s3.getObject({ Bucket: process.env.AWS_S3_BUCKET,
Key: file.name }).promise();
    zip.append(Body, { name: file.name });
  }
  zip.finalize();
});

```

3. `module.exports = router;`

Create `/server/src/services/package.js`:

javascript

Copy

```

const KnowledgeBase = require('../models/knowledgeBase');

const packageKnowledgeBase = async (baseId, metadata) => {
  const base = await KnowledgeBase.findById(baseId);
  base.metadata = metadata;
  await base.save();
  return base;
};

```

4. `module.exports = { packageKnowledgeBase };`

Update `/server/src/index.js`:

javascript

Copy

```

const uploadRoutes = require('../routes/upload');

```

5. `app.use('/api/upload', uploadRoutes);`

6. Test upload/download in Replit using Postman (POST `/api/upload` with multipart form-data, GET `/api/upload/download/:baseId`).

- **Expected Outcome:** APIs for uploading materials and downloading packaged bases.
- **Cohesiveness Notes:** Uploader feeds marketplace; download integrates with purchases.
- **Task 3.3: Blockchain for Badges and Licensing:**
 - **Description:** Extend blockchain for knowledge base licensing.
 - **Files:**
 1. /server/src/contracts/License.sol
 2. /server/src/scripts/deploy.js (update)
 - **Dependencies:** None (uses existing hardhat, ethers).
 - **Steps:**

Create /server/src/contracts/License.sol:

solidity

Copy

```
pragma solidity ^0.8.0;
contract License {
    mapping(address => mapping(string => bool)) public licenses;
    function grantLicense(address user, string memory baseId) public {
        licenses[user][baseId] = true;
    }
}
```

1. }

2. Update /server/src/scripts/deploy.js to deploy License contract.

Update /server/src/resolvers.js:

javascript

Copy

```
const licenseContract = new ethers.Contract('license_contract_address',
['function grantLicense(address,string)'], wallet);
const purchaseKnowledgeBase = async (_, { id, userId }) => {
    // ... existing code ...
    await licenseContract.grantLicense('user_address', id);
    await sendPurchaseEmail('user@example.com', base.title);
    return base;
}
```

3. };

4. Test licensing in Replit or AWS EC2 (npx hardhat console --network ropsten).

- **Expected Outcome:** Blockchain records knowledge base licenses.
- **Cohesiveness Notes:** Extends badges; integrates with marketplace purchases.
- **Task 3.4: Marketplace and Community UI:**
 - **Description:** Enhance UI for uploading materials, packaging, and downloading bases.
 - **Files:**
 1. /client/src/pages/Marketplace.jsx (update)

2. /client/src/components/Uploader.jsx
 - **Dependencies:**
 1. npm: react-dropzone
 - **Steps:**
 1. In /client, run npm install react-dropzone.

Create /client/src/components/Uploader.jsx:

jsx

Copy

```
import React, { useCallback } from 'react';
import { useDropzone } from 'react-dropzone';
import axios from 'axios';
import Card from './Card';
import Button from './Button';

const Uploader = ({ onUpload }) => {
  const onDrop = useCallback(async (acceptedFiles) => {
    const formData = new FormData();
    acceptedFiles.forEach(file => formData.append('files', file));
    formData.append('title', 'New Base');
    formData.append('creator', 'user_id');
    formData.append('subject', 'Math');
    formData.append('price', 10);
    const res = await axios.post('http://localhost:3000/api/upload',
    formData);
    onUpload(res.data);
  }, []);

  const { getRootProps, getInputProps } = useDropzone({ onDrop });

  return (
    <Card className="p-4">
      <div {...getRootProps()} className="border-dashed border-2 p-4
text-center">
        <input {...getInputProps()} />
        <p>Drag & drop files here, or click to select</p>
      </div>
    </Card>
  );
};

2. export default Uploader;
```

Update /client/src/pages/Marketplace.jsx:

jsx

Copy

```
import React, { useState } from 'react';
import { useQuery, gql } from '@apollo/client';
import Card from '../components/Card';
import Button from '../components/Button';
import Uploader from '../components/Uploader';
import axios from 'axios';

const GET_BASES = gql`query { knowledgeBases { id title creator subject price
files { url name } } }`;

const Marketplace = () => {
  const { loading, data } = useQuery(GET_BASES);
  const [uploadedBase, setUploadedBase] = useState(null);

  const handleDownload = async (baseId) => {
    const res = await
    axios.get(`http://localhost:3000/api/upload/download/${baseId}`, {
      responseType: 'blob' });
    const url = window.URL.createObjectURL(new Blob([res.data]));
    const link = document.createElement('a');
    link.href = url;
    link.setAttribute('download', 'knowledge_base.zip');
    document.body.appendChild(link);
    link.click();
  };

  return (
    <div className="min-h-screen bg-secondary p-8">
      <h1 className="text-3xl font-bold text-primary mb-4">Knowledge Base
Marketplace</h1>
      <Uploader onUpload={setUploadedBase} />
      {loading ? <p>Loading...</p> : data.knowledgeBases.map(base => (
        <Card key={base.id} className="mb-4">
          <h2>{base.title}</h2>
          <p>By {base.creator} | ${base.price}</p>
          <Button onClick={() => handleDownload(base.id)}>Download</Button>
        </Card>
      ))}
    </div>
  );
};
```

3. `export default Marketplace;`

4. Run `npm run dev` in `/client` to test UI on Replit's preview.

- **Expected Outcome:** UI for uploading materials and downloading bases.
 - **Cohesiveness Notes:** Reuses Card, Button; integrates with marketplace APIs.
- **Task 3.5: Purchase and Community Emails:**
 - **Description:** Add emails for knowledge base downloads.
 - **Files:**
 1. /server/src/services/email.js
 - **Dependencies:** None.
 - **Steps:**

Update /server/src/services/email.js:

javascript

Copy

```
const sendDownloadEmail = async (email, baseTitle) => {
  await transporter.sendMail({
    from: 'no-reply@platform.com',
    to: email,
    subject: 'Knowledge Base Download',
    html: `<h1>Download Ready!</h1><p>Your base "${baseTitle}" is ready for
download.</p>`,
  });
};
```

1. `module.exports = { /* ... */ sendDownloadEmail };`

Update /server/src/routes/upload.js:

javascript

Copy

```
const { sendDownloadEmail } = require('../services/email');
router.get('/download/:baseId', async (req, res) => {
  // ... existing code ...
  await sendDownloadEmail('user@example.com', base.title);
  zip.pipe(res);
```

2. `});`

3. Test email in Replit's console.

- **Expected Outcome:** Download emails sent, verified via email client.
- **Cohesiveness Notes:** Reuses email service; integrates with marketplace.

Sprint 4: Summer Camps and Analytics Dashboard (3 Weeks)

Tasks:

- **4.1: APIs for Camp Registration/Scheduling** (Updated to use packaged bases).
- **4.2: Predictive Analytics APIs** (Updated to track material usage).
- **4.3: Camp Registration UI and Analytics Dashboard** (No changes).
- **4.4: Camp and Insight Emails** (No changes). **Prompts:**

- **Task 4.1: APIs for Camp Registration/Scheduling:**
 - **Description:** Enhance camp APIs to use packaged knowledge bases.
 - **Files:**
 1. /server/src/models/camp.js (update)
 2. /server/src/routes/camp.js (update)
 - **Dependencies:** None.
 - **Steps:**

Update /server/src/models/camp.js:

javascript

Copy

```
const schema = new mongoose.Schema({
  title: String,
  type: String,
  dates: String,
  ageRange: String,
  theme: String,
  schedule: [{ day: String, activities: [{ time: String, activity: String,
  baseUrl: String } ] }],
  registeredUsers: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  knowledgeBases: [{ type: mongoose.Schema.Types.ObjectId, ref:
'KnowledgeBase' }],

1. });
```

Update /server/src/routes/camp.js:

javascript

Copy

```
router.post('/register', async (req, res) => {
  const { campId, userId, baseUrl } = req.body;
  const camp = await Camp.findById(campId);
  camp.registeredUsers.push(userId);
  if (baseUrl) camp.knowledgeBases.push(baseUrl);
  await camp.save();
  await sendCampRegistrationEmail('user@example.com', camp.title);
  res.json(camp);

2. });

3. Test API in Replit (POST /api/camp/register, body: { "campId": "id",
"userId": "user_id", "baseUrl": "base_id" }).
```

- **Expected Outcome:** Camps use packaged bases for activities.
- **Cohesiveness Notes:** Integrates with marketplace; extends curriculum.
- **Task 4.2: Predictive Analytics APIs:**
 - **Description:** Enhance analytics to track material usage.
 - **Files:**
 1. /ai/src/analytics.py (update)

- **Dependencies:** None.
- **Steps:**

Update /ai/src/analytics.py:

python

Copy

```
def predict_engagement(data):
    base_usage = data.get('base_usage', [])
    recommendation = f"Assign base ID: {base_usage[0]['baseId']}" if
base_usage else 'Explore marketplace'

    1.     return {'risk': 'low', 'recommendation': recommendation,
               'baseId': base_usage[0]['baseId'] if base_usage else null}
```

Update /server/src/routes/analytics.js:

javascript

Copy

```
router.post('/predict', async (req, res) => {
    const base_usage = await KnowledgeBase.find({ purchasedBy: req.body.userId
});
    const python = spawn('python', [path.join(__dirname,
'../../../../ai/src/analytics.py'), JSON.stringify({ ...req.body, base_usage
})]);
    // ... existing code ...

    2. });

    3. Test API in Replit (POST /api/analytics/predict, body: { "userId": "user_id",
        "scores": [80, 90], "engagement": 0.7 }).
        ○ Expected Outcome: Analytics tracks material usage, recommends bases.
        ○ Cohesiveness Notes: Integrates with marketplace; supports tutor
            recommendations.
    ● Tasks 4.3-4.4: No changes.
```

Sprint 5: Discounts, Referrals, and Website Generation (3 Weeks)

Tasks:

- **5.1: APIs for Discount/Referral Creation** (Updated to support marketplace bases).
- **5.2: Blockchain for Referral Rewards/Certificates** (No changes).
- **5.3: APIs for Website Generation** (Updated to include packaged bases).
- **5.4: Discount/Referral and Website UI** (No changes).
- **5.5: Discount/Referral and Website Emails** (No changes). **Prompts:**
- **Task 5.1: APIs for Discount/Referral Creation:**
 - **Description:** Enhance discount APIs to apply to marketplace bases.
 - **Files:**
 1. /server/src/routes/discount.js (update)
 - **Dependencies:** None.

- **Steps:**

Update /server/src/routes/discount.js:

javascript

Copy

```
router.post('/apply', async (req, res) => {
  const { code, userId, baseId } = req.body;
  const discount = await Discount.findOne({ code });
  if (!discount) return res.status(404).json({ message: 'Invalid code' });
  if (baseId) {
    const base = await KnowledgeBase.findById(baseId);
    // Apply discount to base price
  }
  res.json({ message: 'Discount applied' });

```

1. });

2. Test API in Replit (POST /api/discount/apply, body: { "code": "SAVE10", "userId": "user_id", "baseId": "base_id" }).

- **Expected Outcome:** Discounts apply to marketplace base purchases.
- **Cohesiveness Notes:** Integrates with marketplace; extends subscriptions.
- **Task 5.3: APIs for Website Generation:**
 - **Description:** Enhance website generation to include packaged bases.
 - **Files:**
 - 1. /ai/src/website.js (update)
 - **Dependencies:** None.
 - **Steps:**

Update /ai/src/website.js:

javascript

Copy

```
const generateWebsite = async (branding, baseId) => {
  const base = await KnowledgeBase.findById(baseId);
  const prompt = `Generate React code for a branded website with logo:
${branding.logo}, colors: ${branding.colors}, showcasing knowledge base:
${base.title}`;
  const response = await axios.post('https://api.anthropic.com/v1/complete', {
    prompt,
    model: 'claude-3-sonnet',
    max_tokens_to_sample: 500,
  }, { headers: { 'x-api-key': process.env.CLAUDE_API_KEY } });
  return response.data.completion;

```

1. };

Update /server/src/routes/website.js:

javascript

Copy

```
router.post('/generate', async (req, res) => {
  const { branding, baseId } = req.body;
  const websiteCode = await generateWebsite(branding, baseId);
  await sendWebsiteEmail('user@example.com', 'http://generated.website.com');
  res.json({ code: websiteCode });
```

2. });

3. Test API in Replit (POST /api/website/generate, body: { "branding": { "logo": "url", "colors": "blue" }, "baseId": "base_id" }).

- **Expected Outcome:** Websites showcase packaged bases.
- **Cohesiveness Notes:** Integrates with marketplace; extends tutor.
- **Tasks 5.2, 5.4-5.5:** No changes.

Sprint 6: Final Integration and Polish (3 Weeks)

Tasks:

- **6.1: Enhance Tutor APIs for Voice Support** (No changes).
- **6.2: Optimize Community APIs and Add Group Projects** (No changes).
- **6.3: Finalize Analytics with Recommendations** (No changes).
- **6.4: Polish UI for Accessibility** (No changes).
- **6.5: Add Tutor/Community/Insight Emails** (Updated for upload notifications).
- **6.6: Testing and Deployment** (No changes). **Prompts:**
- **Task 6.5: Add Tutor/Community/Insight Emails:**
 - **Description:** Add email for knowledge base upload confirmation.
 - **Files:**
 1. /server/src/services/email.js
 - **Dependencies:** None.
 - **Steps:**

Update /server/src/services/email.js:

javascript

Copy

```
const sendUploadEmail = async (email, baseTitle) => {
  await transporter.sendMail({
    from: 'no-reply@platform.com',
    to: email,
    subject: 'Knowledge Base Uploaded',
    html: `<h1>Upload Complete!</h1><p>Your base "${baseTitle}" is ready for
packaging.</p>`,
  });
};
```

1. module.exports = { /* ... */ sendUploadEmail };

Update /server/src/routes/upload.js:

javascript

Copy

```
router.post('/upload', upload.array('files'), async (req, res) => {  
  // ... existing code ...  
  await sendUploadEmail('user@example.com', req.body.title);  
  res.json(base);  
  
  2. });
```

3. Test email in Replit's console.

- **Expected Outcome:** Upload confirmation email sent.
- **Cohesiveness Notes:** Reuses email service; integrates with uploader.
- **Tasks 6.1-6.4, 6.6:** No changes.